AUTOSAR HLD Document Analysis Assistant Report

A five-stage Streamlit application for structured extraction, grounded question answering, rule-based consistency checking and retrieval benchmarking of AUTOSAR High-Level Design documents

Submitted by

Ruchithra S.

Registration No: 127003217

Department: Computer Science and Engineering

School of Computing

Submission Date: 20.10.2026

Abstract

AUTOSAR High-Level Design (HLD) documents describe software components, ports, interfaces, runnables, basic software dependencies and safety requirements in free-form PDF text, which makes manual review slow and error-prone. This report presents the AUTOSAR HLD Document Analysis Assistant, a Python and Streamlit application that supports such a review in five stages: PDF ingestion and preprocessing with PyMuPDF; structured extraction of AUTOSAR entities with page and evidence provenance; grounded engineering question answering through the Google Gemini API; five rule-based architectural consistency checks; and transparent TF-IDF and Truncated SVD retrieval with a benchmark evaluation studio.

The software was verified by 71 automated unit and integration tests, all of which passed (0 failures, 0 errors, 0 skipped); the Gemini client tests use offline mocks. On the bundled three-page sample HLD, the consistency checker produced 11 findings (8 PASS, 1 WARNING, 0 ERROR, 2 UNVERIFIED). Retrieval was evaluated on 10 curated queries at K = 3 and a similarity threshold of 0.15. Truncated SVD with 4 components achieved mean Precision@3 of 46.67%, Recall@3 of 100.00%, F1@3 of 62.00%, Mean Reciprocal Rank of 0.9500 and retrieval citation accuracy of 46.67%, identical to the TF-IDF baseline; a 2-component SVD scored lower (MRR 0.8333).

The benchmark measures retrieval against expected document pages. It does not evaluate the quality or citation accuracy of generated answers, and a single small sample document does not support claims of enterprise-scale performance or of SVD improving accuracy. The tool is an engineering document analysis assistant and not a certified AUTOSAR compliance or ISO 26262 verification tool.

Keywords: AUTOSAR, High-Level Design, document analysis, information retrieval, TF-IDF, Truncated SVD, Streamlit, Gemini API, consistency checking.

Table of Contents

1. Introduction **5**
1.1 Background 5
1.2 Problem Statement 5
1.3 Motivation 5
2. Objectives and Scope **5**
2.1 Objectives 5
2.2 In-Scope Functionality 6
2.3 Out-of-Scope Functionality 6
3. Technologies and Tools Used **6**
4. System Architecture **7**
4.1 High-Level Architecture 7
4.2 Component Responsibilities 7
4.3 End-to-End Workflow 8
4.4 Data and Evidence Flow 8
5. Detailed Implementation **9**
5.1 PDF Ingestion and Preprocessing 9
5.2 Structured AUTOSAR Extraction 10
5.3 Grounded Engineering Question Answering 10
5.4 Architectural Consistency Checking 11
5.5 Retrieval Transparency and Benchmark Studio 12
5.6 User Interface and Export Functionality 14
6. Testing and Evaluation **15**
6.1 Test Environment and Command 15
6.2 Test Suite Results 15
6.3 Benchmark Methodology 15
6.4 Benchmark Metrics and Results 16
6.5 Interpretation of Findings 17
7. Results and Discussion **18**
8. Limitations and Responsible Use **18**
8.1 Limitations 18
8.2 Responsible Use 19
8.3 Security Notes from the Final Audit 19
9. Future Enhancements **19**
10. Conclusion **20**
11. References **21**
Appendix A: Per-Query Benchmark Results **22**
Appendix B: Ground-Truth Benchmark Dataset **22**
Appendix C: Consistency Findings on the Sample HLD **23**
Appendix D: Data Reconciliation Notes **23**

List of Figures and Tables

List of Figures

Figure 1: High-level architecture and data flow of the AUTOSAR HLD Document Analysis Assistant

Figure 2: Preprocessing and chunking inspector with diagnostics (Retrieval Transparency Studio)9

Figure 3: Structured AUTOSAR entity extraction interface with evidence status10

Figure 4: Architectural consistency checking interface12

Figure 5: Benchmark Evaluation Studio — baseline TF-IDF configuration and scorecard (K = 3, τ = 0.15)13

Figure 6: Benchmark Evaluation Studio — Truncated SVD (2 components) configuration and scorecard14

Figure 7: Main interface of the application (screenshot not supplied — placeholder)14

Figure 8: Query-by-query evaluation breakdown — baseline TF-IDF17

Figure 9: Query-by-query evaluation breakdown — Truncated SVD (2 components)17

List of Tables

Table 1: Technologies and their roles in the project6

Table 2: Source modules and their responsibilities7

Table 3: Provenance information retained at each stage8

Table 4: Consistency rules implemented in consistency_checker.py11

Table 5: Consistency findings on the sample HLD, grouped by rule11

Table 6: Example threshold-based accept/reject decisions (Retrieval_Results.csv)12

Table 7: Benchmark dataset: categories, queries and target pages13

Table 8: Export artefacts available from the application14

Table 9: Automated test suite results (python -m unittest discover -s tests -v)15

Table 10: Evidence files used to verify each benchmark claim16

Table 11: Benchmark results at K = 3, τ = 0.15 (10 queries)16

Table 12: Summary of verified outcomes by stage18

Table 13: Per-query retrieved pages and reciprocal rank for the three configurations22

Table 14: Manually labelled ground-truth benchmark dataset22

Table 15: Complete list of consistency findings on the sample HLD23

1. Introduction

1.1 Background

AUTOSAR (AUTomotive Open System ARchitecture) is a worldwide development partnership that standardises the software architecture of automotive electronic control units (ECUs). Its Classic Platform organises an ECU's software into an application layer of Software Components (SWCs), a Runtime Environment (RTE) that connects them, and Basic Software (BSW) modules such as the communication stack, diagnostics and the operating system [1]. SWCs exchange information through typed **ports** that implement **port interfaces** (for example, Sender-Receiver for data and Client-Server for operation calls). Behaviour inside an SWC is divided into **runnables**, which the RTE activates through events such as timers or data reception, and which are mapped to operating-system (OS) tasks.

Before such a design is implemented, it is usually captured in a High-Level Design (HLD) document. Safety-related functions are additionally classified by an Automotive Safety Integrity Level (ASIL) under ISO 26262 [2], and communication carrying safety-critical data may be protected by End-to-End (E2E) mechanisms such as CRC checks and sequence counters.

1.2 Problem Statement

An HLD is normally delivered as a PDF containing free text, lists and tables. Reviewing it means locating every SWC, port, interface, runnable, BSW dependency and safety statement by hand, and then checking by eye that they agree with each other: that every port references a defined interface, that periodic runnables have a period, that runnables are mapped to tasks, that client-server calls have a timeout, and that an ASIL rating is matched by E2E protection. This work is repetitive and easy to get wrong. A general-purpose chatbot can answer questions about the PDF, but without enforced provenance its answers cannot be traced to a page, and it may state things the document does not say.

1.3 Motivation

The project explores a middle path between manual review and unconstrained AI: an assistant in which every extracted element carries its source page and supporting text, in which architectural checks are deterministic and explainable, in which the language model is explicitly instructed to cite pages and to admit insufficient evidence, and in which the retrieval machinery is open to inspection and measured against a labelled benchmark rather than assumed to work. The assistant is intended as an engineering aid that points a reviewer to the right page and the right question; it is not intended to replace the reviewer.

2. Objectives and Scope

2.1 Objectives

1. Ingest an AUTOSAR HLD PDF and preprocess its text into page-aware chunks, reporting preprocessing statistics (Stage 1).
2. Extract a structured architecture model — SWCs, ports, interfaces, runnables, RTE events, BSW modules and safety entries — with source-page and evidence provenance (Stage 2).
3. Provide a grounded question-answering interface over the document using the Google Gemini API, with page citations and explicit handling of insufficient evidence (Stage 3).
4. Apply five defined rule-based consistency checks to the extracted model and present findings with evidence and recommendations (Stage 4).
5. Implement transparent TF-IDF and optional Truncated SVD retrieval and evaluate retrieval quality against a manually labelled benchmark (Stage 5).
6. Verify the software through an automated test suite and a documented runtime check.

2.2 In-Scope Functionality

- Text extraction and metadata inspection of digital-native (text-layer) PDF files using PyMuPDF.
- Detection of numbered chapters and technical headings; candidate extraction of AUTOSAR entities with Detected / Inferred / Unverified evidence status.
- Gemini-based conversational review grounded in document text and extracted entities (requires a user-supplied API key and internet access).
- Five deterministic rules: AR-PORT-01, AR-RUN-01, AR-RUN-02, AR-IF-01 and AR-SAFE-01.
- TF-IDF retrieval with cosine similarity and a configurable threshold, an experimental Truncated SVD option, and a ten-query information-retrieval benchmark.
- A Streamlit interface with filters, evidence displays and JSON/CSV exports.

2.3 Out-of-Scope Functionality

- Certification of AUTOSAR Classic or Adaptive Platform compliance, ARXML schema validation, RTE generation, or ISO 26262 functional-safety audit. The tool is an engineering document analysis assistant, not a certified compliance tool.
- OCR of scanned, image-only PDFs; cross-document analysis across multi-file architectural repositories.
- Evaluation of the quality, readability or citation correctness of Gemini-generated answers, and measurement of hallucination rate.
- Large-scale or enterprise-corpus evaluation, performance/latency/cost benchmarking, deployment, and user studies.

3. Technologies and Tools Used

The application is written in Python (the README states Python 3.10 or later, tested on Python 3.14). Table 1 lists the dependencies declared in requirements.txt together with their role. Version constraints are reproduced exactly as declared.

Table 1: Technologies and their roles in the project

Component	Declared version	Role in this project
Streamlit	streamlit>=1.30.0	Web user interface in app.py: tabs, sliders, tables, masked key input and downloads [3].
PyMuPDF	pymupdf>=1.23.0	PDF text and metadata extraction in pdf_parser.py [4].
NumPy	numpy>=1.24.0	Numerical core of the TF-IDF vectoriser and cosine-similarity retrieval (README: “pure NumPy vectorization”) [5].
google-genai	google-genai>=2.0.0	Official Google Gen AI SDK used by GeminiClient to call the Gemini API [6].
python-dotenv	python-dotenv>=1.0.0	Loads GEMINI_API_KEY from an optional .env file.
PyYAML	pyyaml>=6.0	Reads config.yaml (system configuration, model defaults, check categories).
pandas	pandas>=2.0.0	Declared dependency; tabular data handling. *Role not described in the README — confirm against source.*
unittest	Python standard library	Test runner for the 71 automated tests [7].

Gemini access uses the google-genai package. The README lists gemini-2.5-flash, gemini-2.5-pro and gemini-1.5-flash as supported model options. Because no generated answers were evaluated in this project, the report does not attribute any result to a particular model version.

4. System Architecture

4.1 High-Level Architecture

The system follows a layered pipeline in which a single Streamlit front end (app.py) drives five processing stages implemented under src/. Figure 1 shows the modules and the data that flows between them. Two paths are distinguished. The **offline path** (blue) — parsing, chunking, section detection, extraction, consistency checking, TF-IDF/SVD retrieval and benchmark evaluation — is deterministic and needs neither an API key nor network access. The **online path** (orange) is confined to Stage 3: a grounded prompt is assembled and sent to the Gemini API through client.py.

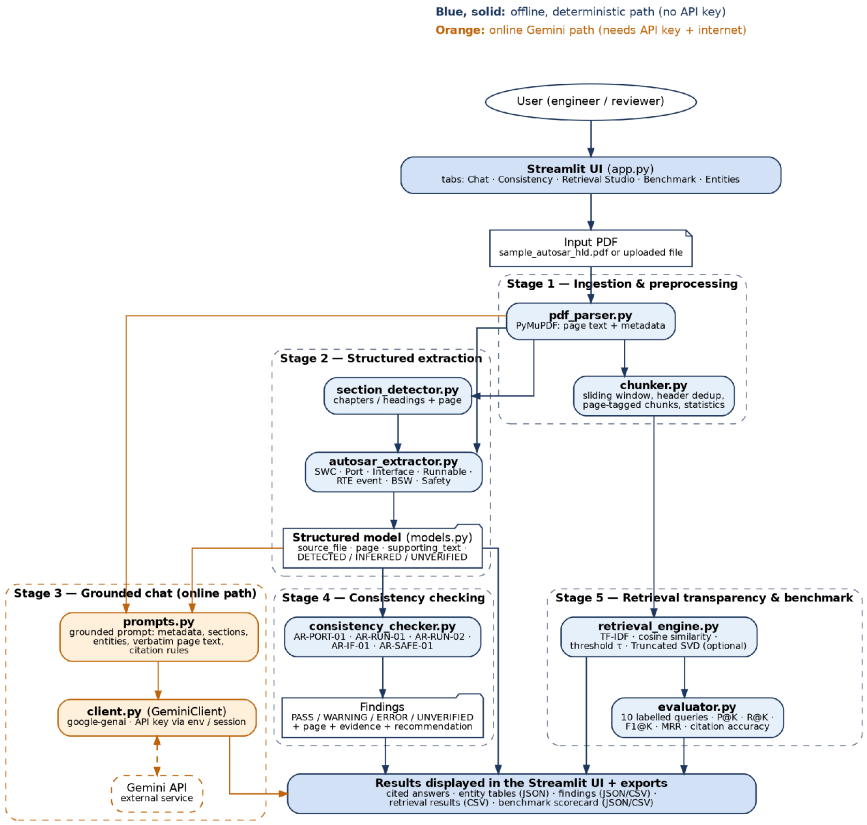


Figure 1: High-level architecture and data flow of the AUTOSAR HLD Document Analysis Assistant

Only connections documented in the project README and visible in its outputs are drawn. In particular, the README describes the Stage 3 prompt as built from document metadata, section structure, structured entities and verbatim page text; it does not describe retrieved TF-IDF chunks being passed to the model, so the diagram shows no link from the retrieval engine to the Gemini path.

4.2 Component Responsibilities

Table 2 maps each source module to its responsibility, following the directory structure documented in the README.

Table 2: Source modules and their responsibilities

Module	Responsibility
app.py	Streamlit entry point; sidebar (document loading, API key), tabs, filters, tables and download buttons.
src/parsers/pdf_parser.py	PyMuPDF text and metadata extraction (page, word and size counts; author/creation metadata; encryption status).

Module	Responsibility
src/parsers/chunker.py	Sanitisation, recurring-header removal and sliding-window chunking with configurable size and overlap; preprocessing statistics.
src/parsers/section_detector.py	Detection of numbered chapters and technical headings with depth and page location.
src/parsers/autosar_extractor.py	Candidate extraction of SWCs, ports, interfaces, runnables/triggers/OS-task mappings, BSW modules and safety/dependency entries.
src/parsers/models.py	Dataclasses EntityType, EvidenceStatus, ExtractionResult carrying provenance.
src/llm/prompts.py	AUTOSARPromptBuilder: grounded prompt with citation mandate and evidence rules.
src/llm/client.py	GeminiClient: API-key handling, calls to Gemini, error handling, mockable for offline tests.
src/analyzers/consistency_checker.py	Five deterministic rules producing severity-tagged findings with evidence and recommendations.
src/analyzers/retrieval_engine.py	TF-IDF vectoriser, cosine-similarity retrieval with threshold, Truncated SVD experiment.
src/analyzers/evaluator.py	Benchmark queries, IRMetricsCalculator and BenchmarkEvaluator.
tests/	Eight unittest modules (71 tests) covering the above.

4.3 End-to-End Workflow

1. The user opens the Streamlit interface and loads the bundled sample HLD (data/input_docs/sample_autosar_hld.pdf) or uploads a PDF.
2. pdf_parser.py extracts text page by page and document metadata with PyMuPDF.
3. chunker.py cleans the text and produces page-tagged sliding-window chunks; section_detector.py locates headings.
4. autosar_extractor.py builds the structured model; every entity records its page, supporting text and evidence status.
5. consistency_checker.py evaluates the five rules over the model and emits findings with page references and recommendations.
6. retrieval_engine.py indexes the chunks (TF-IDF, optionally projected with Truncated SVD) and ranks them for a query, accepting or rejecting each against the threshold τ; evaluator.py repeats this for ten labelled queries and computes the metrics.
7. If the user supplies an API key, the chat tab assembles a grounded prompt through prompts.py and sends it through client.py to Gemini; the answer is displayed with its page citations.
8. Results are displayed in the interface and can be exported as JSON or CSV.

4.4 Data and Evidence Flow

Provenance is carried through the system rather than reconstructed at the end. Table 3 summarises what each stage retains. This design is what allows a consistency finding or a retrieved chunk to be traced back to a page of the source PDF.

Table 3: Provenance information retained at each stage

Stage	Provenance retained
1. Ingestion	Page number for every chunk; chunk identifier; section label; cleaned-text statistics.
2. Extraction	source_file, page_number, supporting_text (verbatim excerpt) and evidence status (DETECTED, INFERRED or UNVERIFIED) for each entity.

Stage	Provenance retained
3. Chat	Citations of the form [Source: <filename>, Page: <page_number>] requested in the prompt, with quotations; the instruction to state insufficient evidence.
4. Consistency	For each finding: rule ID, target entity, page, safety-review flag, message, engineering recommendation and supporting evidence string.
5. Retrieval / benchmark	Rank, status (accepted/rejected), cosine score, threshold, page, section, chunk ID, status reason and chunk excerpt; per-query retrieved and expected pages.

5. Detailed Implementation

Module-level details in this chapter are taken from the project README and from the exported outputs and screenshots supplied for this report. The Python source files themselves were not supplied for inspection; claims that go beyond what the README and outputs show are therefore avoided or marked.

5.1 PDF Ingestion and Preprocessing

Parsing: pdf_parser.py uses PyMuPDF to extract the text layer of each page together with document metadata: page count, word count, file size, author and creation metadata, and encryption status. The interface offers a file uploader, a one-click loader for the bundled sample PDF, a page-by-page reader and a keyword scanner that highlights AUTOSAR terms. Because extraction relies on the text layer, scanned image-only PDFs require prior OCR.

Chunking: chunker.py sanitises the text, removes recurring header/footer lines (optional), and splits the text into overlapping word windows. Window size and overlap are user-configurable; Figure 2 shows the defaults of 120 words and 25 words with header deduplication enabled. The diagnostics panel reports, for the sample HLD, 4 chunks, 308 total words, an average of 83.2 words per chunk, 2,482 cleaned characters and 0 repeated lines filtered. Since 4 × 83.2 ≈ 333 exceeds 308, consecutive chunks evidently overlap. Each chunk is tagged with a single page number (chunk identifiers such as chunk_p2_1 and the page column in the retrieval export confirm this), so page provenance survives chunking.

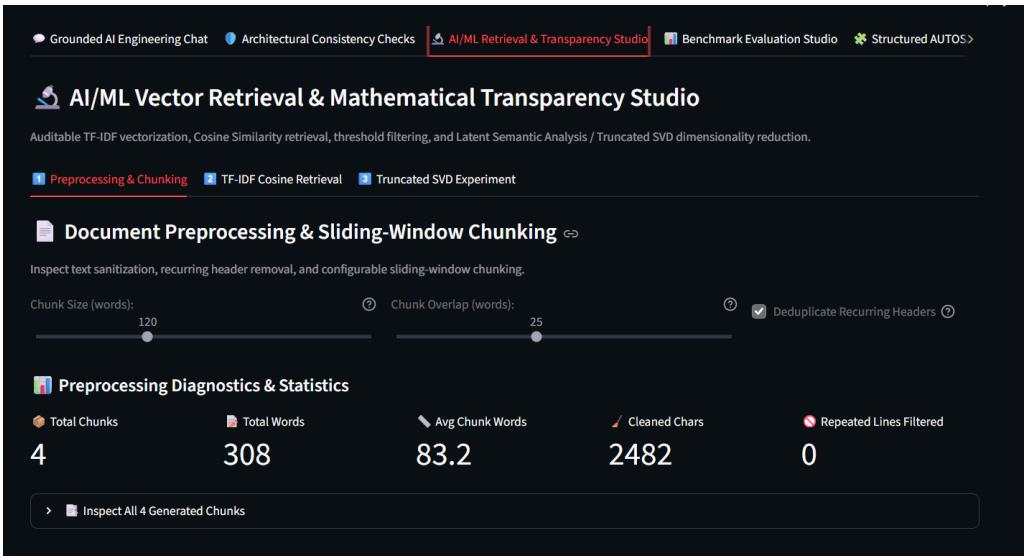


Figure 2: Preprocessing and chunking inspector with diagnostics (Retrieval Transparency Studio)

Figure 2 also shows the top-level tab bar of the application and the three sub-tabs of the Retrieval Transparency Studio: Preprocessing & Chunking, TF-IDF Cosine Retrieval and Truncated SVD Experiment.

5.2 Structured AUTOSAR Extraction

section_detector.py finds numbered chapters (for example “1. ARCHITECTURAL OVERVIEW”) and technical headings with their depth and page. autosar_extractor.py then produces candidate entities of the following kinds: Software Components (target application SWC and connected sensor/actuator SWCs), ports (P-, R- and PR-ports with direction and interface mapping), port interfaces (Sender-Receiver and Client-Server with data elements or operations), internal behaviour (runnables, triggering RTE events, periodicity and OS-task mapping), BSW modules (communication stack, Dem, NvM, OS, MCAL), and safety and dependency items (ASIL rating, E2E profile, checklist status). According to the README, this stage runs entirely without an API key; no language model is involved in extraction.

Every entity is stored with source_file, page_number and a verbatim supporting_text, plus an evidence status: **DETECTED** (stated directly with concrete evidence), **INFERRED** (deduced from naming patterns or surrounding context) or **UNVERIFIED** (mentioned without full specification, or marked under review). The distinction matters downstream: the consistency checker reports unspecified items as UNVERIFIED rather than assuming compliance. For the sample HLD the interface reports 4 SWCs, 3 ports, 3 interfaces, 3 runnables, 3 RTE events and 8 BSW modules (Figure 3). The visible rows illustrate the status model: all four SWC entities are marked as detected, while the attributes of three of them record that the component type was “inferred from naming convention”.

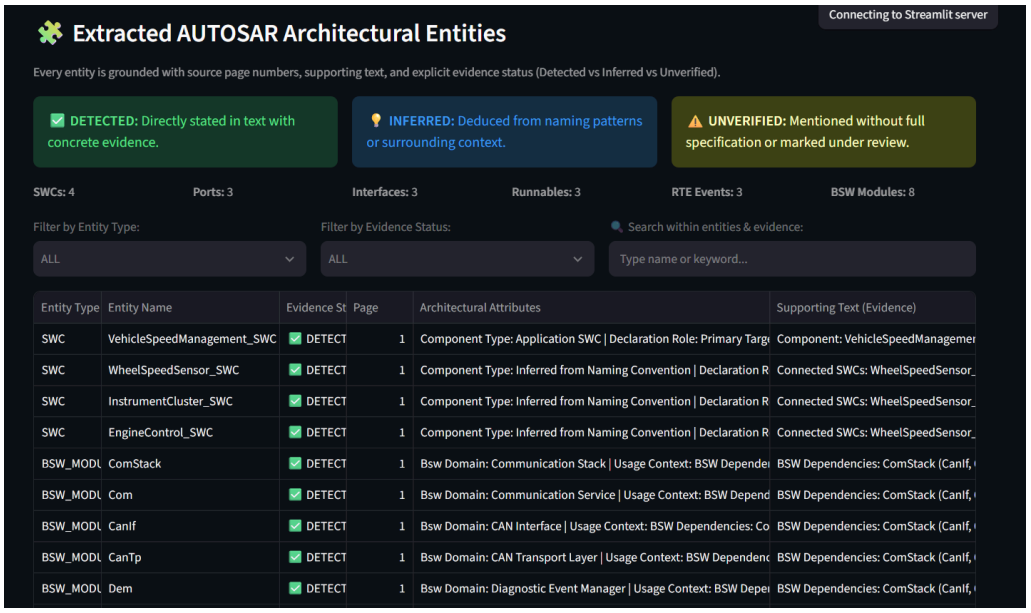


Figure 3: Structured AUTOSAR entity extraction interface with evidence status

The interface provides filters by entity type and evidence status and a free-text search across names, attributes and supporting evidence. The structured model can be exported as JSON (README).

5.3 Grounded Engineering Question Answering

The chat tab lets a reviewer ask architectural questions in natural language, with pre-built question chips for common queries. client.py wraps the Google Gen AI SDK in a GeminiClient; the API key is read from the GEMINI_API_KEY environment variable or entered in a masked input and held only in the Streamlit session, and is never logged or committed. The client handles missing keys, quota errors (HTTP 429), network time-outs and empty responses.

Grounding is implemented in prompts.py by AUTOSARPromptBuilder. According to the README, the prompt combines the document metadata, section structure, structured entities and verbatim page text, and instructs the model to: (i) cite every statement as [Source: <filename>, Page: <page_number>] with quotations; (ii) state explicitly when the evidence is insufficient instead of guessing; (iii) separate explicit document facts from architectural observations; and (iv) never claim standards compliance without

authorised reference rules. All other features, including parsing, extraction, search and export, work without a key.

This is **prompt-level grounding**: it constrains and requests behaviour but cannot guarantee that the model complies. The project does not claim that hallucinations are eliminated or that each answer is correct, and it did not measure answer quality, citation accuracy of generated text, latency or cost. Nine unit tests exercise the client and the prompt builder using offline mocks, so no live Gemini call is made by the test suite. No screenshot of the chat tab was supplied; only its tab label is visible in Figure 2.

5.4 Architectural Consistency Checking

consistency_checker.py evaluates five defined rules over the structured model (Table 4). Each rule yields findings with one of four severities: PASS, WARNING, ERROR or UNVERIFIED. UNVERIFIED is used where the HLD does not provide enough information to decide, for example an unmapped aperiodic runnable or a timeout marked "Under Review".

Table 4: Consistency rules implemented in consistency_checker.py

Rule ID	Name	What is checked
AR-PORT-01	Port-to-interface matching	Each port references a defined interface; unresolved interfaces are flagged.
AR-RUN-01	Runnable trigger and timing	Each runnable has a trigger; periodic runnables have a verified activation period.
AR-RUN-02	Runnable-to-OS-task mapping	Each runnable is mapped to an OS task; undocumented mappings are reported.
AR-IF-01	Client-server operations and timeouts	Client-server interfaces specify operations and an invocation response timeout.
AR-SAFE-01	ASIL and E2E protection	A stated ASIL rating is complemented by an End-to-End protection profile; flagged for safety review.

On the sample HLD the checker produced 11 findings: 8 PASS, 1 WARNING, 0 ERROR and 2 UNVERIFIED (Figure 4; Table 5; the full list is in Appendix C). The single warning concerns IF_DiagStatusService (operation GetFaultStatus), whose client response timeout is marked "Under Review" on page 3 of the HLD; the recommendation is to specify a maximum response timeout so that a client task cannot block indefinitely. The two UNVERIFIED findings record that the OS-task mapping of the aperiodic runnables RE_VSM_Init and RE_DiagEventUpdate is not documented in the HLD. The AR-SAFE-01 finding passes because ASIL-B is accompanied by "Profile 2 (CRC-8 + Sequence Counter)" on IF_VehicleSpeed; its recommendation still asks a functional-safety engineer to confirm the CRC polynomial and counter thresholds.

Table 5: Consistency findings on the sample HLD, grouped by rule

Rule ID	Name	PASS	WARN	UNVER.	Entities (page)
AR-PORT-01	Port-to-interface	3	0	0	ppVehicleSpeed, rpWheelPulseRaw, rpDiagStatus (p. 2)
AR-RUN-01	Trigger and timing	3	0	0	RE_CalculateSpeed, RE_VSM_Init, RE_DiagEventUpdate (p. 2)
AR-RUN-02	Runnable-to-task	1	0	2	RE_CalculateSpeed (pass); RE_VSM_Init, RE_DiagEventUpdate (p. 2)
AR-IF-01	Client-server timeout	0	1	0	IF_DiagStatusService (p. 3)
AR-SAFE-01	ASIL / E2E	1	0	0	ASIL-B (p. 3)

Rule ID	Name	PASS	WARN	UNVER.	Entities (page)
Total		**8**	**1**	**2**	**11 findings, 0 ERROR**

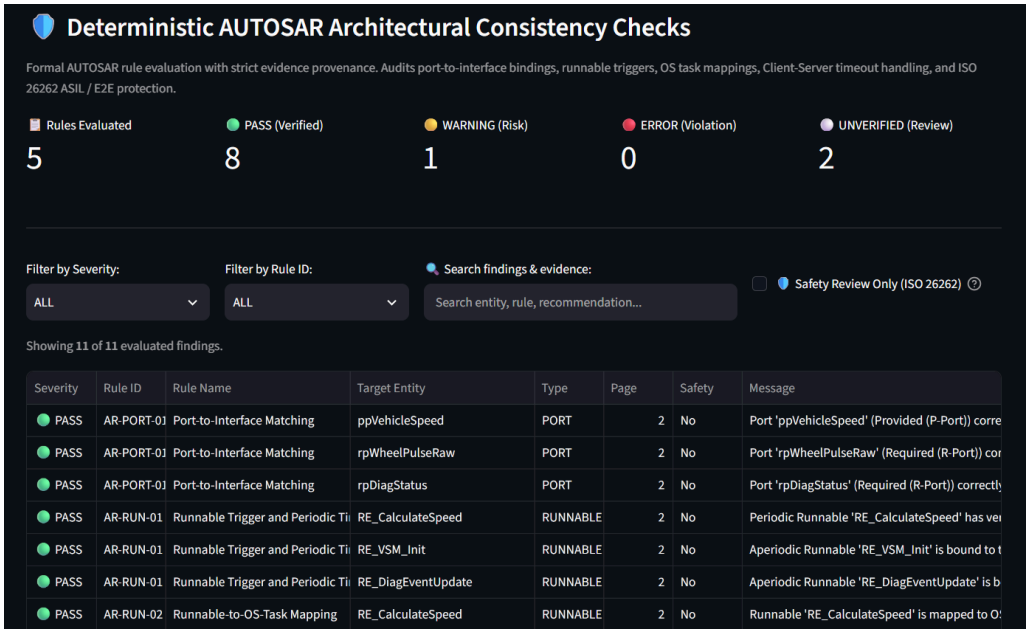


Figure 4: Architectural consistency checking interface

The interface (Figure 4) shows a scorecard (rules evaluated, PASS, WARNING, ERROR, UNVERIFIED), filters by severity and rule ID, a free-text search over findings and evidence, a "Safety Review Only (ISO 26262)" toggle, and a findings table with severity, rule, target entity, type, page, safety flag and message; recommendations and evidence strings are provided in the detail view and in the CSV export. A PASS means that the HLD text contains the evidence the rule looks for. It does not mean the design is correct or compliant. The checker executes five defined rules; it is not a complete AUTOSAR Classic or Adaptive verification, does not replace ARXML validation or RTE-generation toolchains, and does not replace an ISO 26262 audit.

5.5 Retrieval Transparency and Benchmark Studio

Retrieval engine

retrieval_engine.py implements retrieval from first principles so that every step can be inspected. Chunks and queries are converted to term-weighted vectors by a TF-IDF vectoriser written in NumPy, with sublinear term-frequency scaling and a smoothed inverse document frequency (README). In outline, a term's weight in a chunk grows with its (damped) frequency in that chunk and shrinks with the number of chunks containing it, so that rare, specific tokens such as RE_CalculateSpeed dominate over common words [8]. A query is ranked against each chunk by cosine similarity:

$$cos(q, d) = (q \cdot d) / (\|q\| \, \|d\|)$$

A configurable threshold τ separates **accepted** chunks (score ≥ τ) from **rejected** ones, and the interface records a status reason for each. Table 6 reproduces one such export for a single query at τ = 0.15 (the query text is not recorded in the CSV): only the top chunk (cosine 0.2035) is accepted; the others score 0.0829 and 0.0000. The interface also lets the user inspect vocabulary weights.

Table 6: Example threshold-based accept/reject decisions (Retrieval_Results.csv)

Rank	Status	Cosine	Page	Section label (as exported)	Chunk ID
1	ACCEPTED	0.2035	1	BSW Dependencies: ComStack (CanIf, CanTp, Com), Dem, NvM	chunk_p1_1

Rank	Status	Cosine	Page	Section label (as exported)	Chunk ID
2	REJECTED	0.0829	2	RUNNABLES & RTE EVENT MAPPINGS	chunk_p2_1
3	REJECTED	0.0000	2	RUNNABLES & RTE EVENT MAPPINGS	chunk_p2_2
4	REJECTED	0.0000	3	BSW & SAFETY REQUIREMENTS	chunk_p3_4

Truncated SVD experiment

As an experiment, the TF-IDF matrix A can be approximated by keeping only its k largest singular values:

$$A \approx U_k \Sigma_k V_k^T$$

Chunks and queries are then compared in a dense k-dimensional "latent" space instead of the full vocabulary space. This is the idea behind Latent Semantic Analysis [9]; it is intended to let related terms support each other, while its drawback is that distinctive tokens can be blurred. The interface provides a Truncated SVD option and reports latency, dimensionality reduction and explained variance. Truncated SVD is used as the more precise term throughout this report because PCA conventionally involves mean-centred data. Strictly, Truncated SVD and PCA coincide only when the data are mean-centred, so "Truncated SVD" is the more precise term and is used throughout this report [10].

Benchmark Evaluation Studio

The Benchmark Studio evaluates retrieval against ten manually labelled queries grounded in the three-page sample HLD, spread over seven architectural categories (Table 7). Each query has expected target page(s). For a chosen K, threshold τ and method (Baseline TF-IDF or Truncated SVD with a selectable number of components), the studio ranks chunks for every query and computes the metrics defined in Section 6.3. Figures 5 and 6 show the configuration controls and scorecards for the baseline and for a 2-component SVD run.

Table 7: Benchmark dataset: categories, queries and target pages

Category	Queries	Target page(s)	Topic
SWC Architecture	Q01, Q02	1	Primary SWC and type; connected SWCs
BSW Dependencies	Q03	1	ComStack (CanIf, CanTp, Com), Dem, NvM
Ports & Interfaces	Q04, Q05	2	P-port ppVehicleSpeed; R-ports and required interfaces
Internal Behavior	Q06, Q07	2	Runnable triggers, timing event, OS task
Functional Safety	Q08	1 and 3	ISO 26262 ASIL-B; E2E Profile 2
BSW Diagnostics	Q09	3	Condition for Dem_SetEventStatus
Architectural Verification	Q10	3	Checklist item under review

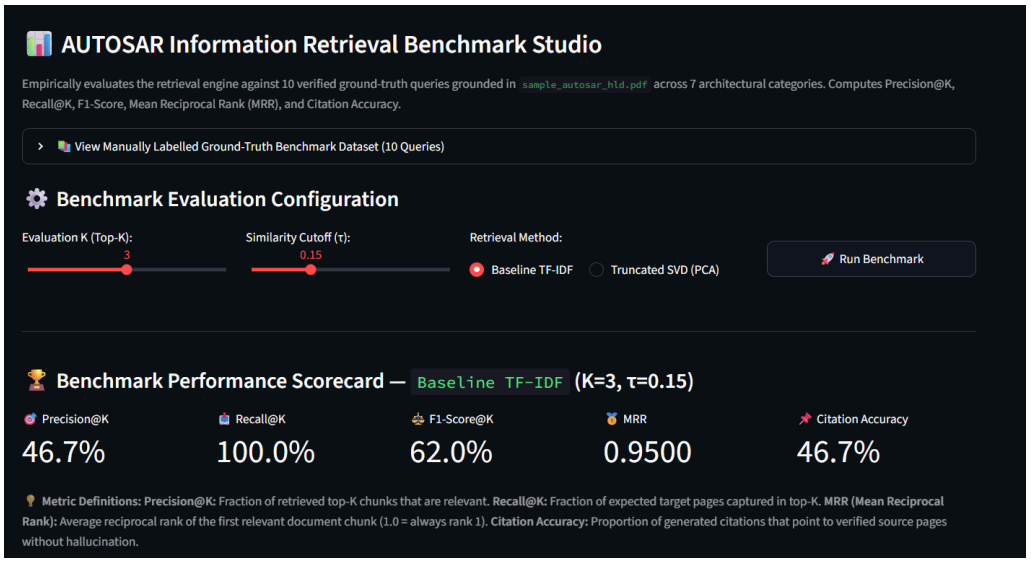


Figure 5: Benchmark Evaluation Studio — baseline TF-IDF configuration and scorecard (K = 3, τ = 0.15)

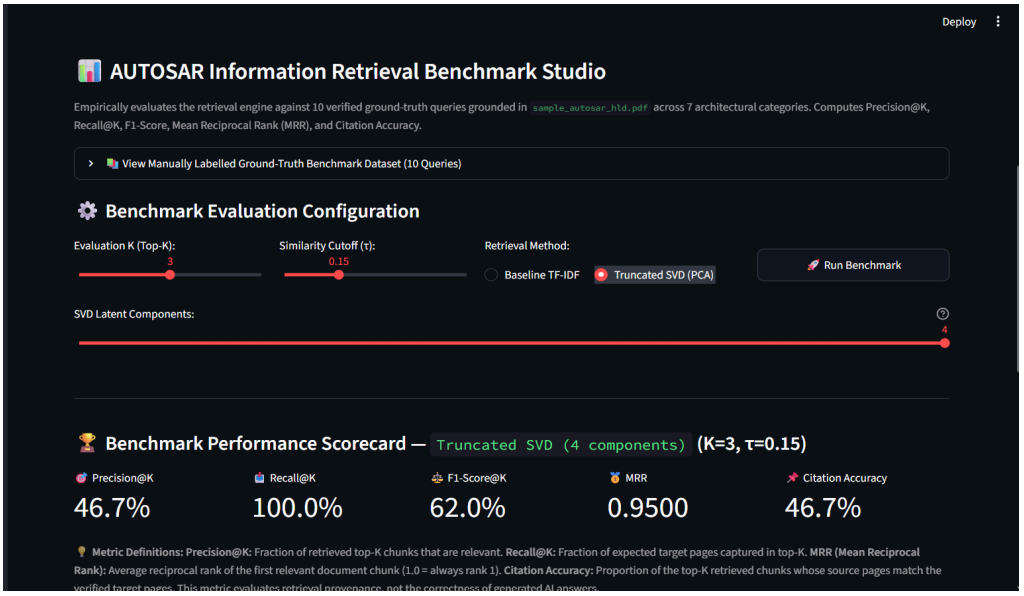


Figure 6: Benchmark Evaluation Studio — Truncated SVD (2 components) configuration and scorecard

5.6 User Interface and Export Functionality

app.py presents the application as a tabbed Streamlit dashboard. The tab bar visible in Figure 2 lists: Grounded AI Engineering Chat; Architectural Consistency Checks; AI/ML Retrieval & Transparency Studio; Benchmark Evaluation Studio; and a fifth tab, truncated in the screenshot, whose page heading reads “Extracted AUTOSAR Architectural Entities” (Figure 3). The tab bar is scrollable, so further tabs may exist beyond the visible area. Document loading and the API-key field are provided in the sidebar (README). Common interaction patterns are filters, free-text search, evidence/provenance columns and sliders for chunk size, overlap, K, τ and SVD components. Table 8 lists the export artefacts.

Table 8: Export artefacts available from the application

Artefact	Format	Evidence
Structured architecture model	JSON	Described in README; not supplied.
Architectural consistency findings	CSV, JSON	CSV supplied (Architectural_Consistency_Checks.csv); JSON per README.

Artefact	Format	Evidence
Retrieval results (rank, score, status, excerpt)	CSV	Supplied (Retrieval_Results.csv).
Manually labelled ground-truth dataset	CSV	Supplied (Manually_Labelled_Ground-Truth_Benchmark_Dataset.csv).
Benchmark scorecard (per-query)	CSV, JSON	CSV supplied for the 2-component SVD run; JSON (Evaluation_Results.json) supplied for the 4-component SVD run.

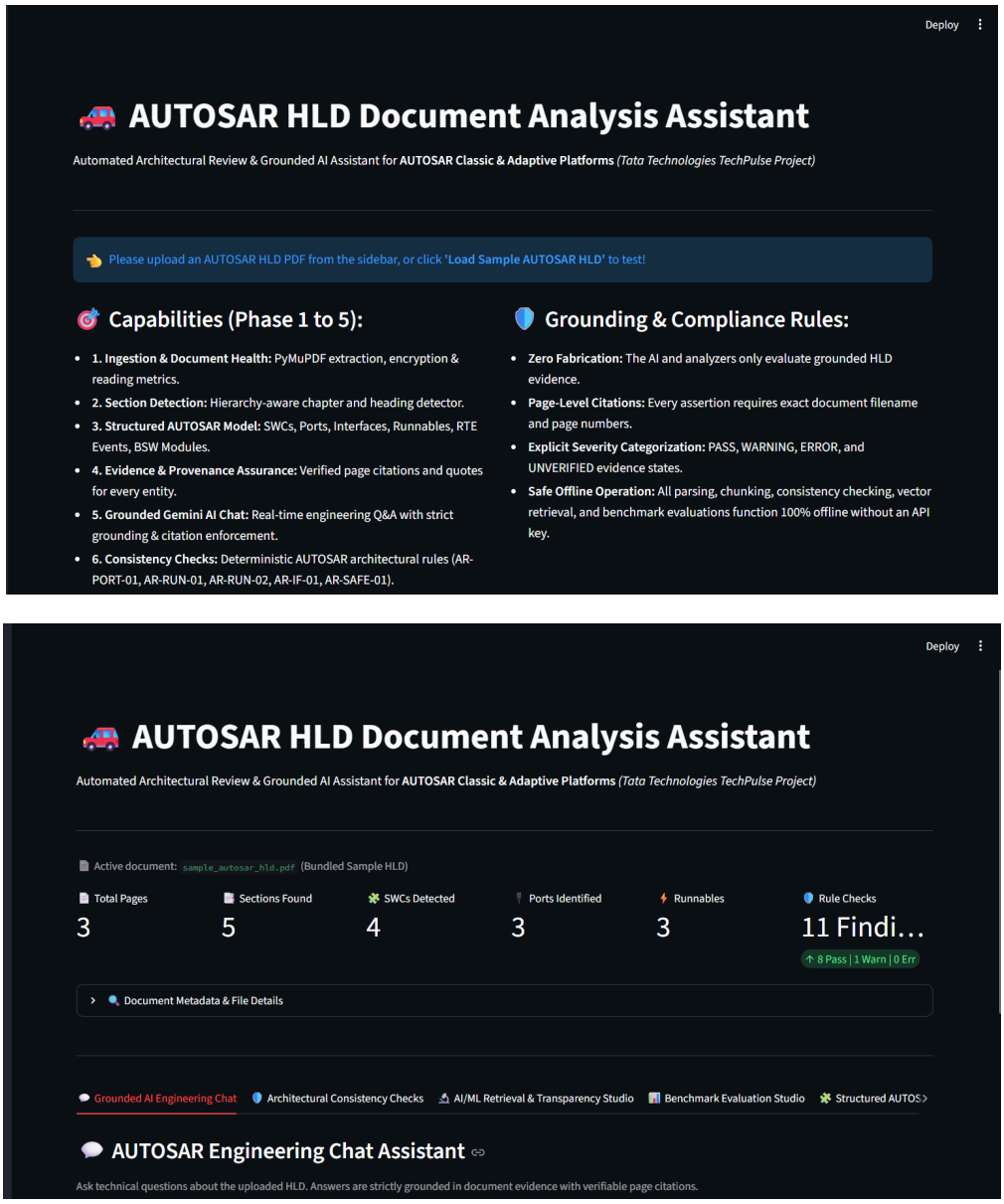


Figure 7: Main interface of the application

6. Testing and Evaluation

6.1 Test Environment and Command

The automated suite is run from the project root with the standard-library test runner:

```
python -m unittest discover -s tests -v
```

The README states that the project targets Python 3.10 or later and was tested on Python 3.14. The final verification audit reported that the application launches with python -m streamlit run app.py and that a headless server on port 8501 returned HTTP 200, after which the process was stopped.

6.2 Test Suite Results

The audit reported 71 tests run, 71 passed, with 0 failures, 0 errors and 0 skipped, in 0.134 seconds. Table 9 gives the breakdown by suite; the counts sum to 71.

Table 9: Automated test suite results (python -m unittest discover -s tests -v)

Test suite	Test module (README)	Tests	Result
Document chunker and preprocessing	test_chunker.py	8	All passed
Deterministic consistency rules	test_consistency_checker.py	13	All passed
IR metrics and benchmark evaluator	test_evaluator.py	8	All passed
Gemini client and prompt grounding (offline mocks)	test_llm_client.py	9	All passed
PyMuPDF parser	test_pdf_parser.py	4	All passed
TF-IDF and SVD retrieval engine	test_retrieval_engine.py	12	All passed
Stage 5 UI component/state integration	test_stage5_ui_integration.py	9	All passed
Structured AUTOSAR model extractor	test_structured_extractor.py	8	All passed
Total		**71**	**71 passed, 0 failed**

Verification boundary. What was verified is that the automated tests pass and that the Streamlit server starts and answers HTTP 200. The Gemini client tests use offline mocks, so the suite does not exercise live Gemini API calls. The audit did not simulate manual in-browser interaction; the screenshots in this report were captured separately by the student from the running interface. The tests establish software correctness against the specified behaviour; they do not establish extraction accuracy on documents other than the sample, or the quality of generated answers.

6.3 Benchmark Methodology

The benchmark measures **information-retrieval quality against expected document pages**. It evaluates how well the retrieval engine ranks chunks, whether the expected pages are covered, and whether the retrieved chunks come from the right pages. Each of the ten queries (Appendix B, Table 14) is labelled with one or two target pages in sample_autosar_hld.pdf. A retrieved chunk is counted as relevant if its page is among the query's expected pages. For K retrieved chunks the metrics are:

$$Precision@K = |relevant\ chunks\ in\ top\text{-}K| \,/\, K$$

$$Recall@K = |expected\ pages\ found\ in\ top\text{-}K| \,/\, |expected\ pages|$$

$$F1@K = 2 \cdot Precision@K \cdot Recall@K \,/\, (Precision@K + Recall@K)$$

$$MRR = (1/|Q|)\ \Sigma_i\ 1/rank_i$$

where $rank_i$ is the position of the first relevant chunk for query i. "Citation accuracy" is defined in the README as the fraction of top-K retrieved chunks that match the verified target pages; it is a statement about retrieved-chunk provenance. Precision, recall and MRR follow standard definitions in the IR literature [8]. Three configurations were run at K = 3 and $\tau = 0.15$: baseline TF-IDF, Truncated SVD with 4 components (the exported JSON) and Truncated SVD with 2 components (the exported CSV and screenshots). Table 10 records which evidence file supports each claim.

Table 10: Evidence files used to verify each benchmark claim

Configuration	Evidence	Verification performed
Truncated SVD, 4 components	Evaluation_Results.json (aggregates and per-query data)	Aggregates recomputed from the 10 per-query records: identical to the stored values.
Truncated SVD, 2 components	Benchmark_Performance_Scorecard___Truncated_SVD.csv; Figures 6 and 9	Recomputed from CSV: P 0.4333, R 1.0, F1 0.5900, MRR 0.8333; matches the on-screen scorecard.
Baseline TF-IDF	Figures 5 and 8 only (no export supplied)	Per-query rows transcribed from Figure 8 and re-scored: P 0.4667, R 1.0, F1 0.6200, MRR 0.9500; matches Figure 5.
Ground-truth dataset	Manually_Labelled_Ground-Truth_Benchmark_Dataset.csv	Expected pages used for all re-scoring above.

What the benchmark does not measure. It does not measure the correctness or readability of Gemini-generated answers, the accuracy of citations produced inside generated answers, the hallucination rate, or generalisation to large enterprise HLD corpora. The ten queries were curated for a three-page sample document, so the benchmark is a focused, small-scale evaluation and not statistically representative evidence of enterprise-scale performance.

6.4 Benchmark Metrics and Results

The exported JSON reports the following for Truncated SVD with 4 components, K = 3 and $\tau = 0.15$ over 10 queries: mean Precision@3 = 0.4667 (46.67%), mean Recall@3 = 1.0000 (100.00%), mean F1@3 = 0.6200 (62.00%), MRR = 0.9500 and retrieval citation accuracy = 0.4667 (46.67%). Table 11 places this result beside the other two configurations.

Table 11: Benchmark results at K = 3, $\tau = 0.15$ (10 queries)

Configuration	Mean P@3	Mean R@3	Mean F1@3	MRR	Citation acc.
Baseline TF-IDF	46.67%	100.00%	62.00%	0.9500	46.67%
Truncated SVD, 4 comp. (JSON)	46.67%	100.00%	62.00%	0.9500	46.67%
Truncated SVD, 2 comp. (CSV)	43.33%	100.00%	59.00%	0.8333	43.33%

The README reports an F1 of 58.67% for the 2-component run. The exported CSV, the on-screen scorecard (59.0%) and a recomputation from the ten per-query F1 values (seven queries at 0.50 and three at 0.80, mean 0.59) all give 59.00%, which is the value used here. Per-query detail for all configurations is given in Appendix A.

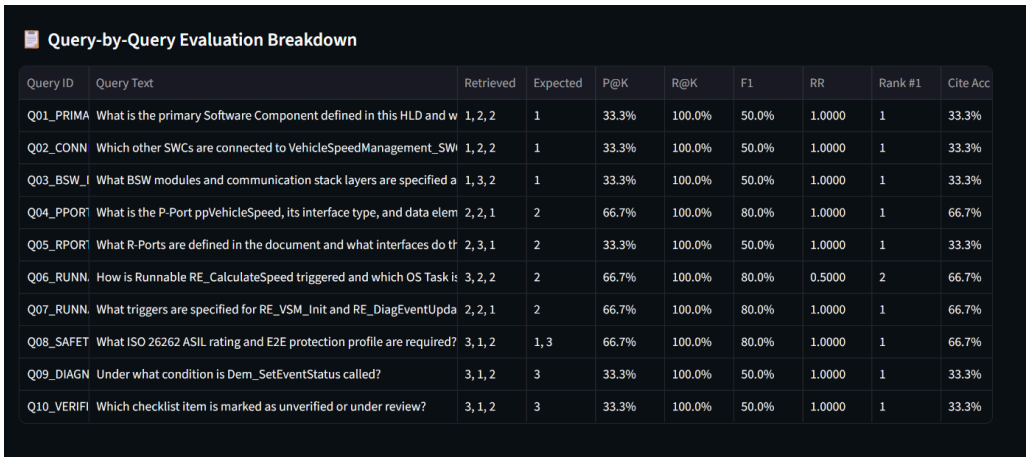
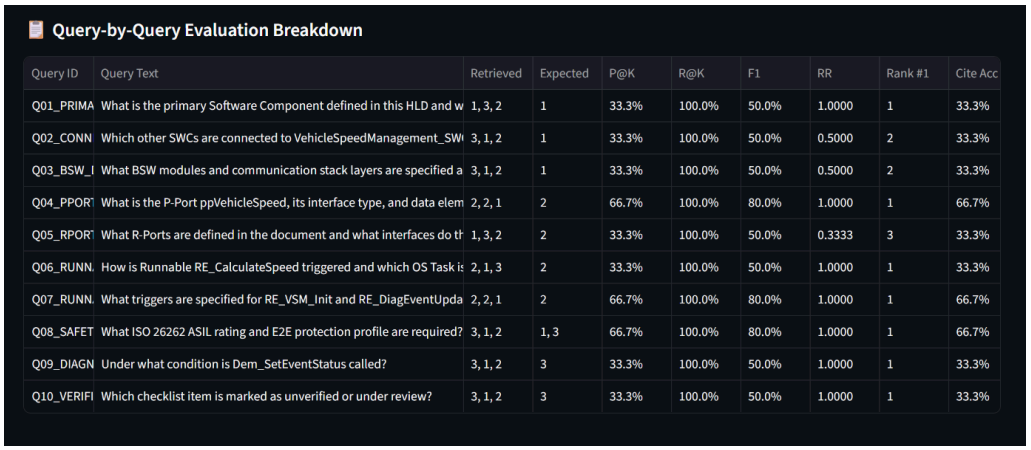


Figure 8: Query-by-query evaluation breakdown — baseline TF-IDF



Query-by-Query Evaluation Breakdown

Query ID	Query Text	Retrieved	Expected	P@K	R@K	F1	RR	Rank #1	Cite Acc
Q01_PRIMA	What is the primary Software Component defined in this HLD and w	1, 3, 2	1	33.3%	100.0%	50.0%	1.0000	1	33.3%
Q02_CONN	Which other SWCs are connected to VehicleSpeedManagement_SW	3, 1, 2	1	33.3%	100.0%	50.0%	0.5000	2	33.3%
Q03_BSW_I	What BSW modules and communication stack layers are specified a	3, 1, 2	1	33.3%	100.0%	50.0%	0.5000	2	33.3%
Q04_PPORT	What is the P-Port ppVehicleSpeed, its interface type, and data elem	2, 2, 1	2	66.7%	100.0%	80.0%	1.0000	1	66.7%
Q05_RPORT	What R-Ports are defined in the document and what interfaces do th	1, 3, 2	2	33.3%	100.0%	50.0%	0.3333	3	33.3%
Q06_RUNN	How is Runnable RE_CalculateSpeed triggered and which OS Task is	2, 1, 3	2	33.3%	100.0%	50.0%	1.0000	1	33.3%
Q07_RUNN	What triggers are specified for RE_VSM_Init and RE_DiagEventUpda	2, 2, 1	2	66.7%	100.0%	80.0%	1.0000	1	66.7%
Q08_SAFET	What ISO 26262 ASIL rating and E2E protection profile are required?	3, 1, 2	1, 3	66.7%	100.0%	80.0%	1.0000	1	66.7%
Q09_DIAGN	Under what condition is Dem_SetEventStatus called?	3, 1, 2	3	33.3%	100.0%	50.0%	1.0000	1	33.3%
Q10_VERIFI	Which checklist item is marked as unverified or under review?	3, 1, 2	3	33.3%	100.0%	50.0%	1.0000	1	33.3%

Figure 9: Query-by-query evaluation breakdown — Truncated SVD (2 components)

6.5 Interpretation of Findings

Ranking is strong; precision is structurally capped. MRR of 0.95 means the first relevant chunk was ranked first for nine of ten queries in the baseline and 4-component runs (only Q06 placed it second). Precision looks modest (46.67%), but it is bounded by the corpus. The sample HLD yields only four chunks — one on page 1, two on page 2 and one on page 3 (Retrieval_Results.csv) — so a query whose only expected page is page 1 or 3 can have at most one relevant chunk among the top three. Under this chunk distribution the maximum achievable mean Precision@3 over the ten queries is 0.50, and the observed 0.4667 reaches about 93% of that ceiling (0.4333 reaches about 87%).

Recall of 100% is largely a property of the setup. With four chunks in total, a top-3 list covers three quarters of the entire corpus, so every expected page is retrieved for every query in all three configurations. Recall at K = 3 therefore does not discriminate between configurations here.

Truncated SVD did not improve retrieval. The 4-component run matched the baseline on every aggregate metric, and the 2-component run was lower on precision, F1 and MRR. Relative to the baseline, the 2-component run lost the first-rank position on Q02, Q03 and Q05 and gained it on Q06, a net MRR change of −0.1167, driven by four queries. On a corpus of four chunks the chunk-by-term matrix has rank at most four, so a 4-component SVD cannot discard any chunk-space variance; this probably explains why it tracks the baseline. These results give no evidence that SVD improves performance on this benchmark, and they cannot show that it would not on a larger corpus. The README's explanation that low-rank SVD loses acronym information is a plausible hypothesis that the benchmark does not test.

Citation accuracy is not an independent metric. In the benchmark exports, citation accuracy equals Precision@3 because both metrics are based on the proportion of the top-three retrieved chunks that match the expected source pages. Therefore, citation accuracy measures retrieval-level source-page matching; it does not evaluate citations generated by Gemini or establish that generated answers are free from hallucinations. The benchmark results should be interpreted within this limited scope.

The threshold appears not to affect the reported metrics. In the JSON, each query lists three retrieved pages even where accepted_chunks_count is 1 or 2 (for example Q02 accepts one chunk but scores three), so Precision@K appears to be computed on the top-K ranking regardless of τ, with τ reported through the accepted-chunk count. This reading is inferred from the exports and should be confirmed in evaluator.py.

7. Results and Discussion

Table 12 summarises what was verified at each stage. Taken together, the results show a working end-to-end prototype in which provenance is preserved from the PDF page to the final finding, checks are deterministic and explainable, and retrieval behaviour is exposed and measured.

Table 12: Summary of verified outcomes by stage

Stage	Verified outcome	Evidence
1. Ingestion	Sample HLD parsed; 4 page-tagged chunks, 308 words, 2,482 cleaned characters.	Figure 2
2. Extraction	4 SWCs, 3 ports, 3 interfaces, 3 runnables, 3 RTE events, 8 BSW modules with page and evidence status.	Figure 3
3. Grounded chat	Client and prompt grounding tested offline (9 tests). Answer quality not evaluated.	Table 9
4. Consistency	5 rules; 11 findings: 8 PASS, 1 WARNING, 0 ERROR, 2 UNVERIFIED.	Figure 4; Table 5
5. Retrieval and benchmark	10 queries at K = 3, τ = 0.15. SVD-4: P@3 46.67%, R@3 100%, F1@3 62.00%, MRR 0.9500; baseline identical; SVD-2 lower.	Table 11
6. Software verification	71 of 71 tests passed; server returned HTTP 200.	Table 9

Two observations illustrate how the stages reinforce each other. First, the one WARNING raised by Stage 4 (the client response timeout marked “Under Review”) is the same item that benchmark query Q10 targets on page 3, and in the benchmark that page ranks first. The retrieval layer can therefore surface the evidence behind a rule finding. Second, the evidence-status model of Stage 2 is what lets Stage 4 distinguish “verified”, “not documented” and “under review” instead of collapsing them into pass or fail; the two UNVERIFIED runnable mappings are honest statements about gaps in the HLD, not detected defects.

The benchmark result is best read as a sanity check of the retrieval pipeline rather than a performance claim. Ranking quality is high on a tiny, curated corpus; precision and recall are constrained by that corpus; and the comparison of TF-IDF with SVD is inconclusive at this scale. The main value of the Benchmark Studio is that these questions can be asked and answered reproducibly with a few controls.

8. Limitations and Responsible Use

8.1 Limitations

1. **Scope of the consistency checker.** Five heuristic rules over one document. It does not prove AUTOSAR Classic or Adaptive compliance, does not parse ARXML, and does not replace schema validation, RTE-generation toolchains or an ISO 26262 audit.
2. **Extraction.** Entities are candidates extracted from text; some attributes are inferred from naming conventions (Figure 3). Only text-layer PDFs are supported, and only one document at a time.
3. **Benchmark size and structure.** Ten curated queries over a three-page, four-chunk document; page-level relevance; recall saturated; precision ceiling of 0.50. Results do not generalise to enterprise corpora and no significance testing was performed.
4. **Threshold and metric semantics.** τ appears not to influence P@K, and citation accuracy duplicates precision (Section 6.5). The scorecard help text in the interface contradicts the metric’s actual definition.

5. **Section labels.** In the retrieval export, chunk chunk_p2_1 begins with the heading "2. PORT INTERFACES & DATA ELEMENTS" yet carries the section label "RUNNABLES & RTE EVENT MAPPINGS", which suggests that labels are assigned at page level rather than per chunk.
6. **Generative component.** Grounding is by prompt instruction only. Answer correctness, citation correctness, hallucination rate, latency and cost were not measured. The chat requires an API key and internet access.
7. **Verification boundary.** The test suite uses offline mocks for Gemini, and manual UI interaction was not simulated by the audit. There was no user study and no deployment.

8.2 Responsible Use

The assistant is a decision-support aid for engineering review. Its outputs, including PASS findings and model answers, must be checked against the source document by a qualified engineer, and safety-related conclusions must go through the project's functional-safety process. Because the Stage 3 prompt contains verbatim page text, using the chat sends document content to an external service; users should consider their organisation's confidentiality rules before loading proprietary HLDs. API keys are entered through a masked input, held in session state and kept out of version control (.env is excluded by .gitignore; .env.example holds only a placeholder).

8.3 Security Notes from the Final Audit

The audit reported that .gitignore excludes caches, virtual environments, editor files and .env; that .env was absent; that .env.example contains a placeholder rather than a real key; and that the sample PDF at data/input_docs/sample_autosar_hld.pdf was preserved. These are audit findings and not a security assessment of the application.

9. Future Enhancements

The following items are **not implemented** and are proposed as future work.

- Retrieval-augmented chat: supply the top-ranked chunks, rather than the full page text, to the Gemini prompt so that the approach scales to HLDs of hundreds of pages.
- Larger and independent benchmark: more queries, longer and multiple documents, labels from more than one annotator, and significance testing, so that TF-IDF and SVD can be compared meaningfully; also evaluate the effect of τ and of chunk size.
- Generative-answer evaluation: measure answer faithfulness, citation correctness and hallucination rate with a labelled set and an agreed method (for example human grading or an LLM-as-judge protocol).
- Richer retrieval: hybrid lexical-plus-embedding retrieval and chunk-level section labelling.
- Broader checks and inputs: additional rules, ARXML import and OCR for scanned PDFs, and multi-document linkage.
- Broader checks and inputs: add further consistency rules, ARXML import, OCR for scanned PDFs and multi-document linkage.

10. Conclusion

This project delivers the AUTOSAR HLD Document Analysis Assistant, a Streamlit application with five implemented stages: PDF ingestion and preprocessing, structured extraction with page-level provenance, Gemini-based grounded question answering, five rule-based consistency checks, and transparent TF-IDF/SVD retrieval with a benchmark studio. All 71 automated tests passed, and the application started successfully with an HTTP 200 response. On the sample HLD the checker returned 11

findings (8 PASS, 1 WARNING, 0 ERROR, 2 UNVERIFIED), and on ten curated queries the 4-component SVD retrieval achieved Precision@3 of 46.67%, Recall@3 of 100.00%, F1@3 of 62.00% and an MRR of 0.9500, identical to the TF-IDF baseline; the 2-component SVD was lower (MRR 0.8333). These are retrieval results on a three-page document and are not evidence of improved accuracy from SVD, of generated-answer quality, or of enterprise-scale performance.

The central design principle — keep the evidence attached to every claim and be explicit about what is unverified — is what the work contributes beyond a plain chatbot. The tool should be read as an engineering document analysis assistant, not as a compliance or certification tool.

11. References

[1] AUTOSAR, “Standards: Classic Platform, Adaptive Platform and Foundation,” AUTOSAR. [Online]. Available: https://www.autosar.org/standards

[2] International Organization for Standardization, ISO 26262-1:2018, “Road vehicles — Functional safety — Part 1: Vocabulary,” 2018. [Online]. Available: https://www.iso.org/standard/68383.html

[3] Streamlit Inc., “Streamlit Documentation.” [Online]. Available: https://docs.streamlit.io

[4] Artifex Software, “PyMuPDF Documentation.” [Online]. Available: https://pymupdf.readthedocs.io

[5] NumPy Developers, “NumPy Documentation.” [Online]. Available: https://numpy.org/doc/

[6] Google, “Gemini API: Libraries (Google Gen AI SDK),” Google AI for Developers. [Online]. Available: https://ai.google.dev/gemini-api/docs/libraries ; SDK source: https://github.com/googleapis/python-genai

[7] Python Software Foundation, “unittest — Unit testing framework,” Python documentation. [Online]. Available: https://docs.python.org/3/library/unittest.html

[8] C. D. Manning, P. Raghavan and H. Schütze, Introduction to Information Retrieval. Cambridge, U.K.: Cambridge University Press, 2008. [Online]. Available: https://nlp.stanford.edu/IR-book/ (term weighting and scoring; evaluation in information retrieval)

[9] S. Deerwester, S. T. Dumais, G. W. Furnas, T. K. Landauer and R. Harshman, “Indexing by latent semantic analysis,” Journal of the American Society for Information Science, vol. 41, no. 6, pp. 391–407, 1990.

[10] scikit-learn developers, “TruncatedSVD,” scikit-learn documentation (concept reference only; this project implements its own NumPy-based retrieval). [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.decomposition.TruncatedSVD.html

References [1]–[10] are external sources describing the tools and concepts used. All descriptions of the project’s own implementation, tests and results are taken from the project README, the verification audit and the exported evidence files supplied by the author.

Appendix A: Per-Query Benchmark Results

Retrieved pages are listed in rank order; the reciprocal rank (RR) of each query is in parentheses. "Expected" gives the ground-truth target page(s). SVD-4 data are from Evaluation_Results.json, SVD-2 data from the exported CSV, and baseline data from Figure 8. All values are at K = 3 and τ = 0.15. Table 13 lists the results.

Table 13: Per-query retrieved pages and reciprocal rank for the three configurations

Query	Expected	Baseline TF-IDF	Truncated SVD (4 comp.)	Truncated SVD (2 comp.)
Q01	1	1, 2, 2 (1.00)	1, 2, 3 (1.00)	1, 3, 2 (1.00)
Q02	1	1, 2, 2 (1.00)	1, 3, 2 (1.00)	3, 1, 2 (0.50)
Q03	1	1, 3, 2 (1.00)	1, 3, 2 (1.00)	3, 1, 2 (0.50)
Q04	2	2, 2, 1 (1.00)	2, 2, 1 (1.00)	2, 2, 1 (1.00)
Q05	2	2, 3, 1 (1.00)	2, 3, 1 (1.00)	1, 3, 2 (0.33)
Q06	2	3, 2, 2 (0.50)	3, 2, 2 (0.50)	2, 1, 3 (1.00)
Q07	2	2, 2, 1 (1.00)	2, 2, 1 (1.00)	2, 2, 1 (1.00)
Q08	1, 3	3, 1, 2 (1.00)	3, 1, 2 (1.00)	3, 1, 2 (1.00)
Q09	3	3, 1, 2 (1.00)	3, 1, 2 (1.00)	3, 1, 2 (1.00)
Q10	3	3, 1, 2 (1.00)	3, 1, 2 (1.00)	3, 1, 2 (1.00)

Mean reciprocal rank: baseline 0.9500; SVD-4 0.9500; SVD-2 0.8333. Mean Precision@3: 46.67%, 46.67% and 43.33% respectively.

Appendix B: Ground-Truth Benchmark Dataset

Table 14 reproduces the manually labelled benchmark dataset exported from the application (reference answers omitted for brevity).

Table 14: Manually labelled ground-truth benchmark dataset

Query ID	Category	Target page(s)	Expected entities
Q01_PRIMARY_SWC	SWC Architecture	1	VehicleSpeedManagement_SWC, Application SWC
Q02_CONNECTED_SWCS	SWC Architecture	1	WheelSpeedSensor_SWC, InstrumentCluster_SWC, EngineControl_SWC
Q03_BSW_DEPENDENCIES	BSW Dependencies	1	ComStack, CanIf, CanTp, Com, Dem, NvM
Q04_PPORT_VEHICLE_SPEED	Ports & Interfaces	2	ppVehicleSpeed, IF_VehicleSpeed, VehicleSpeed_kmh
Q05_RPORTS_SPECIFICATION	Ports & Interfaces	2	rpWheelPulseRaw, IF_WheelPulseData, rpDiagStatus, IF_DiagStatusService
Q06_RUNNABLE_CALCULATE_SPEED	Internal Behavior	2	RE_CalculateSpeed, Task_Periodic_10ms, Timing Event, 10ms
Q07_RUNNABLE_EVENTS	Internal Behavior	2	RE_VSM_Init, swc_init, RE_DiagEventUpdate, Data Received Event
Q08_SAFETY_ASIL_E2E	Functional Safety	1 and 3	ASIL-B, ISO 26262, Profile 2, IF_VehicleSpeed
Q09_DIAGNOSTIC_MONITORING	BSW Diagnostics	3	Dem_SetEventStatus, Dem, 25 km/h
Q10_VERIFICATION_CHECKLIST_GAP	Architectural Verification	3	rpDiagStatus, Server response timeout, Under Review

Appendix C: Consistency Findings on the Sample HLD

Table 15 lists all 11 findings exported from the Architectural Consistency Checks tab; recommendations and full evidence strings are in the exported CSV.

Table 15: Complete list of consistency findings on the sample HLD

Severity	Rule	Target entity	Page	Finding (abridged from the exported CSV)
PASS	AR-PORT-01	ppVehicleSpeed	2	P-port references IF_VehicleSpeed (Sender-Receiver).
PASS	AR-PORT-01	rpWheelPulseRaw	2	R-port references IF_WheelPulseData (Sender-Receiver).
PASS	AR-PORT-01	rpDiagStatus	2	R-port references IF_DiagStatusService (Client-Server).
PASS	AR-RUN-01	RE_CalculateSpeed	2	Periodic; RTE Timing Event, 10 ms.
PASS	AR-RUN-01	RE_VSM_Init	2	Aperiodic; RTE Init Event (swc_init).
PASS	AR-RUN-01	RE_DiagEventUpdate	2	Aperiodic; Data Received Event on rpWheelPulseRaw.
PASS	AR-RUN-02	RE_CalculateSpeed	2	Mapped to Task_Periodic_10ms (Preemptive, Priority 12).
UNVERIFIED	AR-RUN-02	RE_VSM_Init	2	OS-task mapping not documented in the HLD.
UNVERIFIED	AR-RUN-02	RE_DiagEventUpdate	2	OS-task mapping not documented in the HLD.
WARNING	AR-IF-01	IF_DiagStatusService	3	GetFaultStatus specified, but client response timeout is “Under Review”.
PASS	AR-SAFE-01	ASIL-B	3	ASIL-B complemented by E2E Profile 2 (CRC-8 + Sequence Counter) on IF_VehicleSpeed; safety review flagged.

Appendix D: Data Reconciliation Notes

The following differences between supplied materials were found while verifying the numbers in this report. The values used in the report are stated for each.

1. **SVD component counts.** The exported JSON is a 4-component run, whereas the README table lists a 5-component column and the exported CSV and Figures 6 and 9 are a 2-component run. The report presents the three runs separately and uses the JSON as the headline SVD result.
2. **F1 of the 2-component run.** README: 58.67%. Exported CSV, on-screen scorecard and recomputation: 59.00%. The report uses 59.00%.
3. **Baseline evidence.** No baseline JSON or CSV was supplied. Baseline values come from Figures 5 and 8; they were re-scored from the transcribed per-query rows and agree with the on-screen scorecard.
4. **Citation-accuracy description.** The help text in Figures 5 and 6 refers to “generated citations”; the README and the numerical equality with Precision@K indicate a retrieval-provenance metric. The report follows the README.
5. **Source code.** The Python source files were not supplied. Implementation details are taken from the README, the audit and the exported outputs.